



# Chapter 4 — Technology Stack

Every technology in DOIP is chosen for three reasons: it is **free/open-source**, it is **industry-relevant** (worth learning), and it **fits the software-only constraint**. Each table lists the choice, why, and the considered alternative.

## 4.1 Frontend

| Technology                               | Role                 | Why this choice                                                                     | Alternatives considered                                     |
|------------------------------------------|----------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>React 18 + TypeScript</b>             | UI framework         | Dominant ecosystem, strong typing catches bugs in complex dashboard state           | Vue, Svelte, Angular                                        |
| <b>Vite</b>                              | Build tool           | Instant dev server, fast HMR for map-heavy development                              | CRA (deprecated), Webpack                                   |
| <b>MapLibre GL JS</b>                    | Map rendering        | Open-source fork of Mapbox GL; vector tiles, 60fps WebGL rendering, no license fees | Leaflet (weaker perf at 100+ moving markers), Mapbox (paid) |
| <b>OpenStreetMap / OpenFreeMap tiles</b> | Base map data        | Free, global coverage, no API keys required                                         | Google Maps (paid, restrictive)                             |
| <b>Zustand</b>                           | Client state         | Minimal, fast; ideal for high-frequency entity position updates                     | Redux Toolkit (heavier), Jotai                              |
| <b>TanStack Query</b>                    | Server state         | Caching, retries, invalidation for REST data                                        | SWR                                                         |
| <b>Tailwind CSS + shadcn/ui</b>          | Styling & components | Rapid, consistent dark "ops console" theme                                          | MUI, Ant Design                                             |
| <b>Recharts / ECharts</b>                | Dashboard charts     | Declarative charts for logistics, health, analytics                                 | Victory, Chart.js                                           |

| Technology                                       | Role                     | Why this choice                               | Alternatives considered |
|--------------------------------------------------|--------------------------|-----------------------------------------------|-------------------------|
| <b>Mapbox GL Draw (MapLibre-compatible fork)</b> | Polygon/waypoint drawing | Battle-tested draw tools for zones and routes | Custom canvas tooling   |
| <b>Playwright</b>                                | E2E testing              | Cross-browser, reliable WebSocket-aware tests | Cypress                 |

## 4.2 Backend

| Technology                        | Role                   | Why this choice                                                                           | Alternatives considered                     |
|-----------------------------------|------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Python 3.12</b>                | Primary language       | One language across backend, AI, CV, analytics, and simulation — huge integration savings | Node/Go split stack                         |
| <b>FastAPI</b>                    | API framework          | Async-native, Pydantic validation, automatic OpenAPI docs, first-class WebSocket support  | Django (sync-heavy), Flask (less structure) |
| <b>Pydantic v2</b>                | Schemas & validation   | Single source of truth for event/API contracts                                            | dataclasses + marshmallow                   |
| <b>SQLAlchemy 2 + GeoAlchemy2</b> | ORM + geospatial types | Mature PostGIS integration                                                                | Django ORM, raw SQL                         |
| <b>Alembic</b>                    | DB migrations          | Versioned, reviewable schema evolution                                                    | —                                           |
| <b>Uvicorn</b>                    | ASGI server            | Standard high-performance ASGI                                                            | Hypercorn                                   |
| <b>Celery (or ARQ)</b>            | Async job queue        | Video processing, model training, report generation                                       | RQ, Dramatiq                                |
| <b>NetworkX + custom grid</b>     | Graph algorithms       | A* / Dijkstra over operational-area graph                                                 | —                                           |
| <b>Shapely / pyproj</b>           | Geometry math          | Buffering, intersection, distance in planning logic                                       | —                                           |

## 4.3 Data Layer

| Technology                       | Role                            | Why this choice                                                                         |
|----------------------------------|---------------------------------|-----------------------------------------------------------------------------------------|
| <b>PostgreSQL 16</b>             | System of record                | One engine for relational + geo + vector + JSONB event payloads                         |
| <b>PostGIS</b>                   | Geospatial extension            | Industry-standard spatial indexing (GiST), distance/containment queries, GeoJSON output |
| <b>pgvector</b>                  | Vector search                   | RAG embeddings live next to operational data — no separate vector DB to run             |
| <b>Redis 7</b>                   | Pub/Sub + cache + rate limiting | Phase 1–6 event backbone; sub-ms latency                                                |
| <b>Apache Kafka</b><br>(Phase 7) | Event streaming backbone        | Durable, replayable topics; consumer groups for fan-out; the “scale story” migration    |
| <b>MinIO</b>                     | Object storage                  | S3-compatible local storage for videos, annotated frames, model artifacts               |

## 4.4 AI / ML Stack

| Technology                                   | Role              | Why this choice                                                                                                   | Alternatives                            |
|----------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <b>Ollama</b>                                | Local LLM runtime | Free, offline, private; one-line model serving                                                                    | vLLM (heavier), hosted APIs (cost)      |
| <b>Llama 3 8B / Qwen 2.5 7B</b>              | LLM models        | Strong summarization/instruction-following at consumer-hardware size                                              | Mistral 7B                              |
| <b>LangGraph</b>                             | AI orchestration  | Explicit, debuggable graphs for report pipeline & RAG agent                                                       | LangChain chains, custom code           |
| <b>sentence-transformers (bge/e5 models)</b> | Embeddings        | Free local embeddings for RAG                                                                                     | OpenAI embeddings (cost)                |
| <b>pgvector HNSW</b>                         | Vector index      | Fast approximate nearest-neighbor inside Postgres                                                                 | FAISS, Qdrant, Chroma                   |
| <b>Ultralytics YOLOv8/v11</b>                | Object detection  | SOTA speed/accuracy, trivial fine-tuning, huge community. ⚠️ AGPL-3.0 license — fine because DOIP is open-source; | YOLOX, RT-DETR (Apache-2.0), Detectron2 |

| Technology                      | Role                                 | Why this choice                                                     | Alternatives         |
|---------------------------------|--------------------------------------|---------------------------------------------------------------------|----------------------|
|                                 |                                      | Apache-2.0 alternatives (YOLOX, RT-DETR) documented in Ch. 19 §19.4 |                      |
| <b>OpenCV</b>                   | Video/frame processing               | Frame sampling, drawing annotations                                 | —                    |
| <b>XGBoost</b>                  | Tabular forecasting/classification   | Best-in-class for tabular telemetry                                 | LightGBM             |
| <b>Prophet</b>                  | Time-series forecasting              | Handles trend/seasonality with tiny code footprint                  | statsmodels<br>ARIMA |
| <b>PyTorch (optional)</b>       | LSTM sequence models                 | Only if LSTM experiments are pursued in Phase 6                     | TensorFlow           |
| <b>MLflow (lightweight use)</b> | Model registry & experiment tracking | Versioned models with metrics per Chapter 2 FR-10.4                 | Custom tables        |

## 4.5 DevOps & Infrastructure

| Technology                      | Role                                | Why this choice                                                          |
|---------------------------------|-------------------------------------|--------------------------------------------------------------------------|
| <b>Docker + Docker Compose</b>  | Local dev & Phases 1–6 deployment   | One-command full-stack startup:<br><code>docker compose up</code>        |
| <b>Kubernetes (k3s/kind)</b>    | Phase 7 orchestration               | Lightweight local K8s demonstrates real orchestration without cloud cost |
| <b>GitHub Actions</b>           | CI/CD                               | Free for public repos: lint → test → build → push images → deploy        |
| <b>Nginx</b>                    | Reverse proxy / TLS / static assets | Standard edge; WebSocket proxying                                        |
| <b>Prometheus</b>               | Metrics collection                  | Scrapes FastAPI /metrics, Redis, Postgres exporters                      |
| <b>Grafana</b>                  | Dashboards & alerting               | Ops dashboards for the platform itself (meta-observability)              |
| <b>Loki (optional)</b>          | Log aggregation                     | Structured JSON logs searchable next to metrics                          |
| <b>pre-commit + Ruff + mypy</b> | Code quality                        | Formatting, linting, typing enforced pre-push                            |
| <b>pytest + pytest-asyncio</b>  | Backend testing                     | Async test support for WS and engine tests                               |

## 4.6 Hardware Requirements (Software-Only Confirmation)

| Profile         | Spec                                               | What runs                                                                |
|-----------------|----------------------------------------------------|--------------------------------------------------------------------------|
| Minimum dev     | 8 GB RAM, 4 cores, no GPU                          | Full stack with 7B LLM quantized (Q4) at reduced speed; YOLO nano on CPU |
| Recommended dev | 16 GB RAM, 6+ cores, any NVIDIA GPU ( $\geq 6$ GB) | Comfortable full stack; real-time-ish YOLO; fast LLM inference           |
| Demo/deploy     | One VPS or the same laptop                         | Docker Compose or k3s single node                                        |



**Zero external hardware. Zero paid services.** Every component above runs locally and free. The only optional paid upgrade is swapping Ollama for a hosted LLM API if higher answer quality is wanted for demos. Chapter 19 contains the full zero-cost stack audit, the license-by-license review (including the YOLO AGPL decision), and the complete project cost sheet.

## 4.7 Version Pinning Policy

- Pin exact versions in `requirements.txt` / `package.json` lockfiles; upgrade deliberately once per phase.
- Model weights (YOLO, LLM, embeddings) are versioned artifacts in MinIO with checksums — a run is reproducible end to end.
- One `.env.example` documents every configuration variable; no secrets in the repo.